

Ames Housing Sale Price Estimator

End-to-End Machine Learning Walkthrough Report

Part 1: Setup & Data Exploration

Step 1: Install Required Libraries

```
pip install numpy pandas matplotlib seaborn scikit-learn xgboost optuna shap reportlab pillow joblib
```

Purpose of Key Libraries:

Library	Use Case
numpy, pandas	Core numerical and dataframe operations
matplotlib, seaborn	Visualization and EDA
scikit-learn	Preprocessing, modeling, evaluation
xgboost, optuna	Advanced modeling and tuning
shap	Model explainability via SHAP
joblib	Save/load Python objects (e.g., models)

Step 2: Import Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
```

Step 3: Load the Dataset

```
Ames = pd.read_csv('AmesHousing.csv')
df = pd.DataFrame(Ames)
```

Step 4: Initial Data Exploration

```
df.columns
```

```
df.nunique()
```

Check for feature types, low-variance columns, and inspect categorical values:

```
for i in df.select_dtypes(include='object').columns:
```

```
    print(df[i].value_counts())
```

Step 5: Determine Problem Type

```
df['SalePrice'].value_counts()
```

- The target variable SalePrice is continuous.
 - This is a **regression** problem.
-

Step 6: Dataset Structure Inspection

```
df.info()
```

This provides:

- Column data types
 - Missing value counts
 - Memory usage
-

Part 2: Data Cleaning

Step 7: Missing Value Diagnosis

```
missing_values = df.isnull().sum()
```

```
missing_values_filtered_sorted = missing_values[missing_values > 0].sort_values()
```

```
percent_missing_values = (missing_values_filtered_sorted / df.shape[0]) * 100
```

Columns with over 50% missing values are dropped:

```
['Fireplace Qu', 'Mas Vnr Type', 'Fence', 'Alley', 'Misc Feature', 'Pool QC']
```

Step 8: Drop Irrelevant Columns

```
df.drop(columns=[
```

```
'Order', 'PID',  
'Fireplace Qu', 'Mas Vnr Type', 'Fence', 'Alley', 'Misc Feature', 'Pool QC'  
, inplace=True)
```

Explanation:

- Order, PID are non-informative identifiers.
 - Others had excessive missingness.
-

Step 9: Remove Duplicates

```
df.duplicated().any()  
df.drop_duplicates(inplace=True)
```

Part 3: Imputation & Outlier Removal

Step 10: Split Features and Target

```
X_df = df.drop(columns='SalePrice')  
y_df = df['SalePrice']  
  
Ensure matching indices after dropping duplicates:  
X_df.drop_duplicates(inplace=True)  
y_df = y_df.reindex(X_df.index)
```

Step 11: Train-Test-Validation Split

```
from sklearn.model_selection import train_test_split  
  
X_train, X_val, y_train, y_val = train_test_split(X_df, y_df, test_size=0.15, random_state=42)  
X_train, X_test, y_train, y_test = train_test_split(X_train, y_train, test_size=0.15, random_state=42)
```

Step 12: Handle Missing Values

- **Numerical Columns:**

```
sim_imputer_numeric = SimpleImputer(strategy='median')
```

- **Categorical Columns:**

```
sim_imputer_category = SimpleImputer(strategy='most_frequent')
```

Step 13: Update Imputed Data

Make sure to preserve original indices after transforming:

```
X_train_numeric = pd.DataFrame(sim_imputer_numeric.transform(...))
```

```
X_train_category = pd.DataFrame(sim_imputer_category.transform(...))
```

Step 14: Check for Post-Imputation Duplicates

```
X_train_numeric.duplicated().any()
```

Step 15: Outlier Detection with IQR

```
Q1 = X_train_numeric.quantile(0.25)
```

```
Q3 = X_train_numeric.quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
X_train_numeric_inliers_df = X_train_numeric[~((X_train_numeric < (Q1 - 1.5 * IQR)) |  
                                              (X_train_numeric > (Q3 + 1.5 * IQR)))].any(axis=1)]
```

Repeat for validation and test sets.

Step 16: Sync Other DataFrames

```
X_train_category_inliers_df = X_train_category.reindex(X_train_numeric_inliers_df.index)
```

```
y_train_inliers_df = y_train.reindex(X_train_numeric_inliers_df.index)
```

Step 17: Rebuild Full Feature Sets

```
X_train = pd.concat([X_train_numeric_inliers_df, X_train_category_inliers_df], axis=1)
```

Final Shapes:

- X_train: (805, 73)
 - X_val: (166, 73)
 - X_test: (143, 73)
-

Part 4: Feature Engineering & Modeling

Step 18: Feature Selection

```
important_features = ['Overall Qual', 'Gr Liv Area', 'Garage Area',  
                      'Year Built', 'BsmtFin SF 1', 'Garage Cars', 'Full Bath']  
X_train_7_features = X_train[important_features]
```

Step 19: Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()  
  
X_train_scaled = pd.DataFrame(  
    scaler.fit_transform(X_train_7_features),  
    index=X_train.index,  
    columns=important_features  
)
```

Apply transform() for validation and test sets.

Step 20: Model Training

```
from sklearn.pipeline import Pipeline  
  
from sklearn.linear_model import LinearRegression  
  
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, AdaBoostRegressor  
  
from sklearn.tree import DecisionTreeRegressor  
  
from sklearn.svm import LinearSVR
```

Pipelines Created:

```
model_pipeline = {  
    'LinearRegression': Pipeline([('model_1', LinearRegression())]),  
    ...  
}
```

Step 21: Evaluation Metrics

- **R²** – Explained variance
 - **MSE** – Squared error
 - **MAE** – Real-world interpretability
-

Step 22: Results Summary

Model	R ²	MSE	MAE
Linear Regression	0.88	410M	15,319
Decision Tree	0.74	904M	19,211
Random Forest	0.90	353M	13,832
Gradient Boosting	0.92	292M	12,851
XGBoost	0.89	378M	14,563
AdaBoost	0.87	440M	16,775
LinearSVR	-9.07	Very High	Very High

Conclusion:

Gradient Boosting Regressor was the top-performing model.

Part 5: Model Optimization with Optuna

Objective

Use **Optuna** to optimize the Gradient Boosting Regressor.

Optimization Logic

```
def objective(trial):
```

```
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 1000),
        ...
    }
```

- 50 parameter sets were tested.

- Best model chosen using RMSE as objective.

Final Optimized Model

```
optimal_gbr_mod = GradientBoostingRegressor(loss='squared_error', **best_params)
```

```
optimal_gbr_mod.fit(X_train, y_train)
```

Performance:

- R^2 : 0.92
- RMSE: 19,660.70

Part 6: Explainability & Deployment

Step 24: Feature Importance Visualization

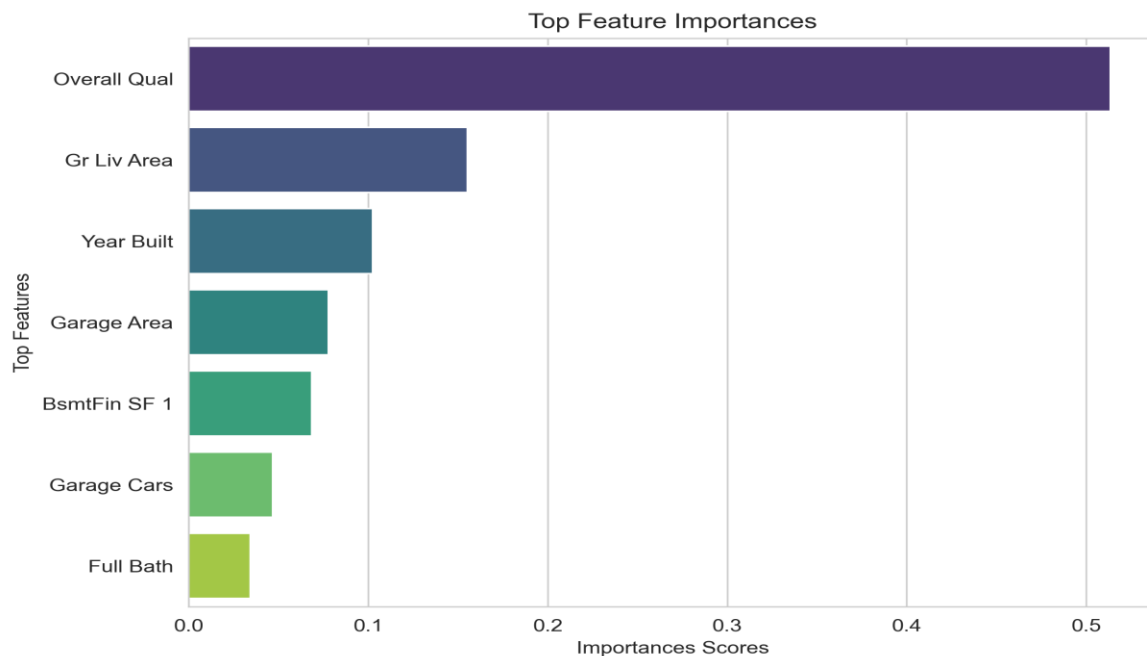
Code:

```
def plot_feature_importance_type1(model, feature_names, top_n=5):
```

```
    ...
```

```
plot_feature_importance_type1(optimal_gbr_mod, X_train.columns, top_n=7)
```

Output:



A horizontal bar plot of top 7 most important features.

Step 25: SHAP Explanations

```
explainer = shap.Explainer(optimal_gbr_mod, X_train)
shap_values = explainer(X_test)
shap.plots.waterfall(shap_values[0])
```

Interpretation:

- Blue = reduces prediction
 - Red = increases prediction
 - SHAP supports **model transparency**.
-

Step 26: Saving the Model & Scaler

```
sio.dump(optimal_gbr_mod, './Ames_Sale_Price_Model.skops')
joblib.dump(scaler_7_features, './scaler.joblib')
```

Output Files:

- Ames_Sale_Price_Model.skops
- scaler.joblib

These files are essential for web deployment (e.g., Flask app).

Final Summary Table

Step	Description
Data Cleaning	Dropped nulls, irrelevant columns, duplicates
Imputation	Median and most frequent value strategies
Outlier Removal	IQR method
Feature Engineering	Top 7 features selected + scaling
Model Training	7 models tested with cross-validation
Model Selection	Gradient Boosting Regressor chosen
Optimization	Optuna tuning of GBR
Explainability	Feature importance + SHAP visualization
Deployment	Saved model and scaler for Flask integration

Walkthrough of app.py: House Sale Price Estimator Web App (Flask Backend)

This document provides a detailed, step-by-step explanation of the Flask-based app.py backend that powers a **House Sale Price Estimator Web App**. The app allows users to input property data, receive a predicted house sale price, and understand the model's reasoning via SHAP explanations.

1. Purpose of the File

The app.py script defines the backend logic of the web app using **Flask**, a lightweight Python web framework. It handles:

- Receiving and processing user input from a form or API.
 - Making predictions using a pre-trained machine learning model.
 - Visualizing SHAP-based explanations for model transparency.
 - Rendering HTML templates like index.html and shap.html.
-

2. Importing Libraries

```
import skops.io as sio
from joblib import load
import shap
import numpy as np
import pandas as pd
from flask import Flask, request, render_template, jsonify, url_for, session, redirect
from flask_cors import CORS
import matplotlib.pyplot as plt
import base64
import io
import os
import warnings
from sklearn.exceptions import InconsistentVersionWarning
```

Key Libraries Explained

- **Flask:** Powers web routing, session handling, and rendering HTML templates.
 - **shap, matplotlib, numpy, pandas:** For model interpretability and visualizations.
 - **skops.io, joblib:** Load the pre-trained model and scaler.
 - **base64, io:** Convert plots to embeddable image formats for web display.
-

3. App Configuration

```
app = Flask(__name__)
```

```
app.secret_key = '1234'
```

```
CORS(app)
```

- `Flask(__name__)`: Initializes the web server.
 - `secret_key`: Enables session storage to maintain user input between routes.
 - `CORS(app)`: Allows requests from other origins (e.g., a separate frontend).
-

4. Loading the Trained Model and Scaler

```
trusted_types = [...] # Whitelisted components required for secure model loading
```

```
model = sio.load('./Ames_Sale_Price_Model.skops', trusted=trusted_types)
```

```
scaler = load('./scaler.joblib')
```

- `model`: The optimized Gradient Boosting Regressor trained on historical housing data.
 - `scaler`: The StandardScaler object used to normalize input data prior to prediction.
 - These files were saved during the model training phase in the Jupyter notebook.
-

5. Feature Definitions

```
feature_names = [
```

```
    "Overall Qual", "Gr Liv Area", "Garage Area",
```

```
    "Year Built", "BsmtFin SF 1", "Garage Cars", "Full Bath"
```

```
]
```

These are the **7 features** selected for model input. The order and format must exactly match the training data schema.

6. Homepage Route

```
@app.route('/')  
  
def home():  
    image_url = url_for("static", filename="images/Ames_image.jpeg")  
    return render_template("index.html", image_url=image_url)
```

Explanation

- Responds to a browser request at /.
 - Loads index.html, the main page where the user inputs data.
 - Passes in the background image for visual appeal.
-

7. Prediction Route

```
@app.route('/predict', methods=['POST'])  
  
def predict():  
    ...
```

What Happens in This Route:

1. **Receive JSON Input from Frontend**
2. `data = request.get_json()`

Example input:

```
{ "Gr Liv Area": 1900, "Garage Area": 450, ... }
```

3. **Convert to DataFrame**
4. `input_data = [float(data.get(feats, 0)) for feats in feature_names]`
5. `input_df = pd.DataFrame([input_data], columns=feature_names)`
6. **Scale the Inputs**
7. `input_scaled_df = scaler.transform(input_df)`
8. **Make a Prediction**
9. `prediction = model.predict(input_scaled_df)[0]`
10. **Save Inputs for SHAP Analysis**
11. `session['last_input'] = pd.DataFrame(input_scaled_df, columns=feature_names).to_json()`
12. **Return Prediction as JSON**

```
13. return jsonify({
14.     'prediction': round(prediction, 2),
15.     'shap_link': url_for('shap_page')
16. })
```

8. SHAP Explanation Route

```
@app.route('/shap')
def shap_page():
    ...
```

SHAP Workflow:

1. **Load Previous Input from Session**
2. `input_df = pd.read_json(session.get('last_input'))`
3. **Compute SHAP Values**
4. `explainer = shap.Explainer(model)`
5. `shap_values = explainer(input_df)`
6. **Generate SHAP Waterfall Plot**
7. `shap.plots.waterfall(shap_values[0], show=False)`
8. **Convert Plot to Web-Ready Image**
9. `buf = io.BytesIO()`
10. `plt.savefig(buf, format='png', bbox_inches='tight')`
11. `image_base64 = base64.b64encode(buf.getvalue()).decode('utf-8')`
12. **Render the HTML Page**
13. `return render_template("shap.html", shap_image=image_base64)`

The SHAP explanation visually communicates how each input feature influenced the final prediction, aiding interpretability.

9. Running the Web Application

```
if __name__ == '__main__':
    app.run(debug=True)
```

- Starts the Flask development server on `http://127.0.0.1:5000/`.
- The application can now accept inputs, predict house prices, and explain predictions using SHAP.

Summary for Beginners

Section	Description
<code>import ...</code>	Loads all necessary Python libraries
<code>app = Flask(...)</code>	Sets up the web server and session capabilities
<code>model = sio.load(...)</code>	Loads the trained Gradient Boosting model
<code>@app.route('/')</code>	Serves the homepage with the input form
<code>@app.route('/predict')</code>	Handles user input, processes it, returns a predicted house price
<code>@app.route('/shap')</code>	Generates and displays a SHAP explanation for the prediction
<code>app.run(...)</code>	Runs the Flask application locally

Frontend Walkthrough Report

Overview

This guide provides a structured explanation of the frontend components of the **Ames House Sale Price Estimator** web application. It includes two files:

- `index.html` – The user interface for input and prediction.
- `shap.html` – Displays SHAP explanations for model interpretability.

These HTML files interact with a Flask backend to collect input, trigger predictions, and display model interpretation plots.

1. `index.html` – Prediction Page

Purpose

- Collects user input for house features.
 - Sends the input to the Flask backend.
 - Displays the predicted sale price.
 - Provides access to a SHAP explanation.
 - Includes a loading spinner during processing.
-

Section-by-Section Breakdown

1.1 HTML Document Setup

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Ames House Sale Price Estimator</title>
```

- Declares a standard HTML5 page.
 - `lang="en"` sets the language to English.
 - Title appears in the browser tab.
-

1.2 Page Styling (Embedded CSS)

```
body {  
    background-image: url("{{ image_url }}");  
    background-size: cover;  
    ...  
}
```

- `{{ image_url }}` is dynamically injected from Flask using Jinja templating.
 - A centered `.container` box holds the form.
 - Form elements are styled for usability.
 - `#loading`: A hidden spinner that appears during backend processing.
-

1.3 Data Input Form

```
<form id="prediction-form">  
    <label>Overall Quality Rating: <input name="Overall Qual" ...></label>  
    ...  
    <button type="submit">Predict</button>  
</form>
```

- Collects values for **7 model features**:
 - Overall Qual
 - Gr Liv Area
 - Garage Area
 - Year Built
 - BsmtFin SF 1
 - Garage Cars
 - Full Bath
 - Each input has validation attributes (e.g., min, max).
 - Form is submitted using JavaScript to prevent page reload.
-

1.4 Output & Spinner Containers

```
<div id="result"></div>
```

```
<div id="shap-plot"></div>
```

```
<div id="loading" style="display:none">...</div>
```

- #result: Displays prediction results.
 - #shap-plot: Reserved for future enhancements.
 - #loading: Spinner shown while loading the SHAP page.
-

1.5 JavaScript Logic

```
form.addEventListener('submit', async (e) => {  
  e.preventDefault();  
  ...  
});
```

Functionality Flow:

1. **Form Submission**
 - Prevents page reload with e.preventDefault().
2. **Data Extraction**
 - Uses FormData to collect inputs.
 - Converts form entries into a JSON object.
3. **Sending Request to Backend**
4. const response = await fetch('/predict', {
5. method: 'POST',
6. headers: { 'Content-Type': 'application/json' },
7. credentials: 'include',
8. body: JSON.stringify(data)
9. });
10. **Handling the Response**
11. const result = await response.json();
12. document.getElementById('result').innerHTML = `
13. <h2>Predicted Sale Price: \$\$\${result.prediction.toLocaleString()}</h2>
14. <button id="shap-btn">View SHAP Explanation</button>

15. `;

16. SHAP Button Handler

17. document.getElementById('shap-btn').addEventListener('click', () => {

18. document.getElementById('result').innerHTML = '';

19. document.getElementById('loading').style.display = 'block';

20. window.location.href = result.shap_link;

21. });

22. Error Handling

- Displays console errors if prediction fails.

Summary Table

Section	Purpose
<form>	Collects 7 input values from the user
fetch('/predict')	Sends data to Flask backend
response.json()	Receives predicted price and SHAP URL
shap-btn	Triggers navigation to explanation page
#loading	Displays spinner during redirection
background-image	Dynamically loaded using Flask and Jinja

Beginner Insight

This HTML file interacts with the backend to perform predictions. It does **not** run any machine learning in the browser. JavaScript helps capture form data, talk to Flask, and display results.

2. shap.html – SHAP Explanation Page

Purpose

- Displays a SHAP waterfall plot generated by the backend.
 - Helps users understand the contribution of each feature to the prediction.
 - Provides navigation back to the homepage.
-

Section-by-Section Breakdown

2.1 Basic HTML Structure

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>SHAP Explanation</title>
```

```
</head>
```

- Minimal page setup.
 - Title shown in the browser tab.
-

2.2 Main Content Area

```
<body>
```

```
  <h1>SHAP Feature Contribution</h1>
```

- A heading to indicate the interpretability purpose of the page.
-

2.3 Conditional Rendering with Jinja

```
{% if shap_plot %}
```

```
  
```

```
{% else %}
```

```
  <p>No prediction input available for SHAP explanation.</p>
```

```
{% endif %}
```

How it works:

- If a SHAP plot is generated (as a base64 image), it is embedded directly into the page using an `<img>` tag.
- If not, the user is shown a fallback message.
- Jinja2 templating is used to insert the image or error message.

2.4 Navigation Link

```
<a href="{{ url_for('home') }}">Back to Home</a>
```

- Uses Flask's `url_for` to create a reliable link to the homepage.
- Improves user experience by allowing easy navigation.

Summary Table

Section	Purpose
<code>{% if shap_plot %}</code>	Embeds SHAP plot as an image
<code><img src="data:image..."</code>	Shows the SHAP plot without saving to a file
<code>{% else %}</code>	Displays fallback if no input is available
<code>url_for('home')</code>	Adds navigation back to the form page

End-to-End Workflow Recap

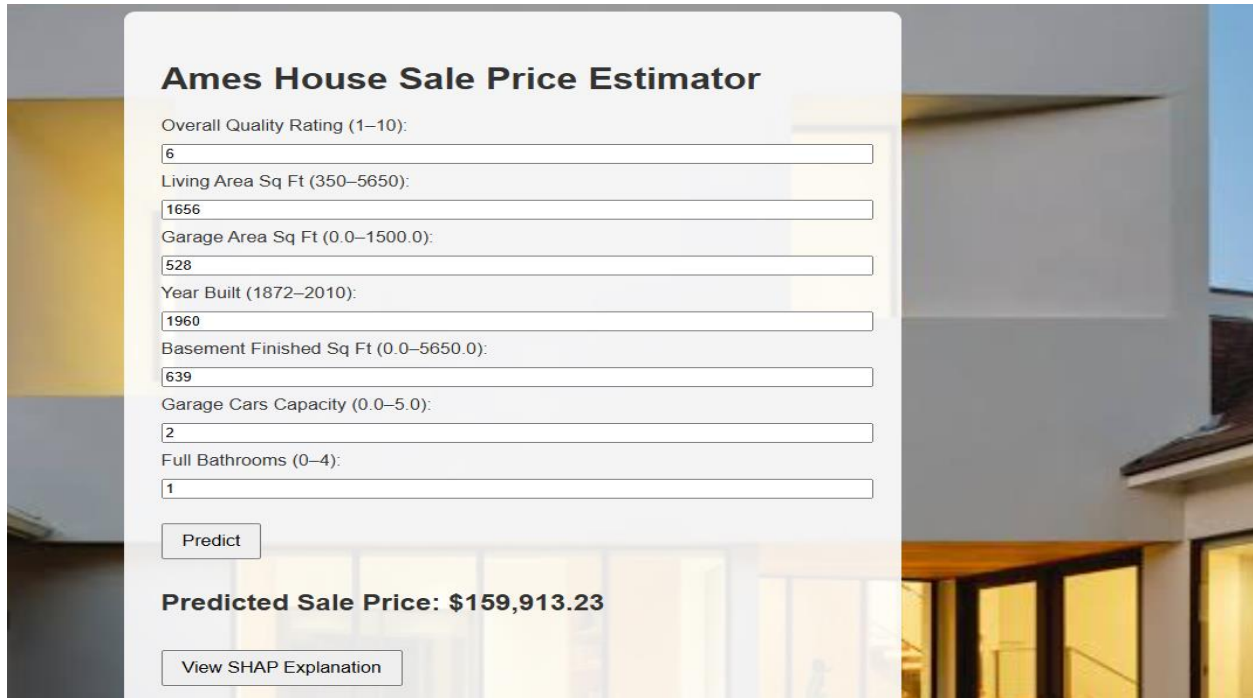
1. **User opens** the web app (`index.html`) and fills in the house feature form.
2. **Form data is sent** to the Flask backend using JavaScript.
3. **Backend predicts** sale price and saves input to the session.
4. **User clicks** “View SHAP Explanation”.
5. **Backend computes** SHAP values and generates a plot.
6. **Plot is encoded** in base64 and passed to `shap.html`.
7. **`shap.html` renders** the image directly in the browser.

Conclusion

Together, `index.html` and `shap.html` create a smooth and interactive user experience:

- The **prediction page** handles data input and interaction.
- The **explanation page** visualizes the logic behind the model.

Both files are lightweight, responsive, and designed to work seamlessly with a Python-Flask backend.



Ames House Sale Price Estimator

Overall Quality Rating (1–10):

Living Area Sq Ft (350–5650):

Garage Area Sq Ft (0.0–1500.0):

Year Built (1872–2010):

Basement Finished Sq Ft (0.0–5650.0):

Garage Cars Capacity (0.0–5.0):

Full Bathrooms (0–4):

Predicted Sale Price: \$159,913.23

SHAP Feature Contribution

